Brush engines as a vocabulary for an all-brush graph

This is exploratory research, not a decision to implement a graph or to reproduce another application's engine. Painter currently stores immutable document-space sampled strokes with per-point weight; seeded pencil splats are derived from those strokes and rasterized into a tiled cache. There is no brush graph. The product aspiration that **all** brushes be graph-defined should be tested against brushes that read the canvas or carry paint, not merely against a hard round stamp. In this fragment, **documented** means behavior claimed by an application's own manual or source; **proposed** means a possible Painter design. A resemblance of marks is a **visual approximation**, not a compatible simulation or a promise that another application's presets will load. [1]–[13]

What existing brush editors actually expose

- **Photoshop, documented:** Shape Dynamics varies size, angle and roundness with jitter, fade or pen controls; Color Dynamics can vary once per stroke or per tip; Transfer varies opacity and flow independently. Scattering is a separate Brush Settings category for distributing primary brush marks, while Texture combines a pattern with the brush, optionally per tip. Dual Brush combines two tips with marks at their intersection, with its own spacing, scatter, count and blend mode: ordinary scattering and the second tip's scatter must not be conflated. Mixer Brush is a distinct feedback case: a reservoir deposits paint, a pickup well continually samples canvas color, and Wet, Load and Mix govern pickup, depletion and the canvas/reservoir balance. Its airbrush-style stationary build-up is another time-dependent behavior. These are documented UI behaviors, not published Photoshop implementation equations. [1][2][3][13]
- **Krita, documented:** Its brush editor exposes multiple different engines (including Pixel, Color Smudge, Spray, Bristle and Clone), rather than one universal stamp engine. Sensors include pressure, tilt, speed, drawing angle, distance, time, per-dab and per-stroke randomness. Color Smudge has distinct smearing (sample previous brush position) and dulling (sample under current dab), with color rate and spacing changing the result; Clone samples an identified source layer or projection. Spray chooses particle number and angular/radial distributions. These engine boundaries argue against assuming that changing a tip image is sufficient to make every brush. [4][5][6][7][8]
- **MyPaint/libmypaint, documented:** Brush-setting metadata separates inputs (pressure, random, stroke progress, direction, tilt, fine/gross speed, barrel rotation, etc.) from outputs such as logarithmic radius, hardness, opacity, dabs per radius and dabs per second. Its opacity-linearize setting explicitly compensates for overlapping dabs; dabs per second is not equivalent to distance spacing. The project's engine documentation describes low-pass filtering of velocity. This is evidence for explicit sensor mappings and time/overlap semantics, not for importing MyPaint presets into Painter. [9][10]
- **Procreate, documented:** Brush Studio separates Stroke Path, Shape, Grain, Rendering, Wet Mix, Color Dynamics and Pencil controls. Moving grain travels with a stroke; Texturized grain remains fixed behind it. Wet Mix exposes dilution, charge (paint available at stroke start), attack and pull. Procreate Dual Brush combines two independently editable brushes via Combine Mode—different from Photoshop's intersection of primary and secondary tip marks. Pressure/tilt and per-stamp versus per-stroke color variation are distinct controls. Its 3D metallic/roughness material brushes do not imply that a 2D color dab is a physical material model. [11][12]

Proposed typed vocabulary (not existing Painter APIs)

A useful graph would specify **when** and **where** a value exists, as well as its numerical type. A scalar pressure value at a sample, a mask at a mark and an image snapshot at a document revision are different ports. These candidate types make feedback, spatial texture and reproducibility inspectable:

- `Sample`: one ordered, document-space input observation `(position, time?, pressure?, tilt?, azimuth?, rotation?, device, stroke-id, ordinal)`; missing capabilities are explicit, not synthetic stylus measurements. `StrokeSamples`: ordered input sequence, distinct from an emitted `StrokeRecord` and from a resampled stream. Existing Painter points and weights supply position and weight, not all of these proposed fields.
- `SampleStream` → `DabEventStream`: a path normalizer/stabilizer, arc-length resampler, distance/time emitter or taper can consume ordered samples and emit events `(position, tangent, size, index, time?, seed)`. "Time" cannot be recovered from position-only historical strokes. Stable stroke IDs and seeded per-event random streams would preserve deterministic replay and prefix behavior; streaming and whole-stroke taper may disagree about the final end until pen-up.
- `Control<T, domain>`: constants, sensor curves, clamped combinations, noise (seeded per stroke or mark), fade by distance/time, and low-pass filters yield values such as `Length`, `Angle`, `UnitFraction`, `Color` or `Rate`. Domains are `per-sample`, `per-dab`, `per-stroke`, `per-pixel`, and `per-document`; crossing domains needs named sampling or aggregation. `size` (geometry), `flow` (per-dab delivery), `opacity` (stroke or compositing cap) and `density` (marks per distance/time/area) are separate artist controls. Krita and MyPaint explicitly distinguish several of these. [1][5][9]
- `TipShape` / `MaskField`: analytic disk/ellipse, sampled tip image, bristle bundle or noise field; a tip's mask is not itself deposited paint. `TextureField`: paper fixed in document coordinates, texture in tip-local coordinates, or moving grain with a stroke-coordinate phase; mark-level versus stroke-level masking remains an explicit choice. `DabEventStream + TipShape + Control<...>` → `MarkStream`: positioned colored/covered marks with bounded footprint, orientation, optional blend intent and stable order. A `Mark` is one deposit; a `StrokeRecord` owns the input and graph/preset revision for regeneration, not merely the visible pixels. [2][11]
- `ImageRead`: a **chosen** immutable source image/projection/revision with sampling transform and pixel/color-space policy; `SampleImage(ImageRead, coordinates)` → color or neighborhood. `MaterialState`: proposed pigment load, wetness or bristle state carried between ordered dabs, with explicit initial state, reset and versioning. `Deposit(MarkStream, MaterialState?, ImageRead?)` → an ordered color/alpha/erase operation stream and perhaps new material state. `Composite(previous-image, operations)` → `ImageResult` for tile display. An image result is a rendered observation, not a vector stroke or a writable material. Clone and wet color pickup cannot be modeled as a pure color choice from the input sample alone. [3][6][7][12]

Proposed invariant: a graph must declare if its output is local and replayable from one stroke plus fixed assets, or depends on earlier document/image state. The former can rederive marks when a tile is rendered at any resolution; the latter needs a well-defined prior-stroke snapshot or ordered replay, and possibly a stored checkpoint. A clone sampling "all visible" cannot read its own progressively changing destination unless feedback order is made explicit. Procreate's materials are 3D surface channels; do not silently equate those channels with Painter's proposed 2D wet-pigment state. [6][7][11]

Choosing graph granularity, without pretending the choice is settled

- **Monolithic engine node:** `PixelBrush(samples, settings)` / `SmudgeBrush(samples, image, settings)` is a fast route to credible engine-specific behavior and simple preset UX; internal equations, intermediate previews and composition across engines remain opaque. Krita's different engines illustrate the practical payoff and ceiling. If these nodes become exceptions to "all brushes graph-defined," they should themselves be subgraphs or explicitly acknowledged as native primitives. [4]

- **Macro/subgraph:** `Ink`, `Pencil`, `Spray`, `WetMixer`, `Clone` expose artist controls over a typed assembly of shared emitters, masks, color mapping, feedback and deposit operations. This offers approachable presets plus editable internals but makes preset-version migration and macro port stability consequential. Photoshop's grouped settings and Procreate's Brush Studio suggest useful artist-facing groupings, not identical implementations. [1][2][11]
- **Small primitives:** sample filter, sensor curve, spacing emitter, noise, scatter, tip transform, mask intersection, texture lookup, source sampler and compositor are highly reusable and inspectable; graph sprawl, illegal domain combinations, per-dab overhead and accidental nondeterminism become artist and engineering costs. Keep low-level nodes behind macros for normal painting; show typed ports and scoped previews when editing them.
- **Shader-like pixel field:** a color/coverage function evaluated for a region can express continuous masks, textured fills and resolution-independent detail, but pixels alone cannot express temporal reservoir depletion, fixed drawing order, or source-image dependencies. A shader can be an implementation of `MaskField` or `Deposit`, not necessarily the graph's only data model. The renderer can fuse pure operations without requiring the artist to author shaders. No source establishes a universal granularity winner.

Candidate mappings: paths, controls, and the honest boundary

Each following path is a **proposal**. "Visual" means a plausible similar-looking mark under stated conditions; "simulation" would require a specified reference algorithm, assets, color management, tablet data and side-by-side conformance evidence not provided here. Adobe, Krita, MyPaint and Procreate describe settings, not interchangeable formulas. [1]–[12]

- **Hard round / solid ink:** `samples → stabilize? → distance-spacing → pressure-curve(size) → disk or nib mask → color → deposit`. Expose size, hardness, smoothing, pressure curve, flow and stroke opacity; use a directional elliptical nib and begin/end taper for calligraphy. For a pure hard brush this is a strong visual starting point, not a claim to Photoshop's opacity accumulation or Procreate's stabilization equation. [1][5][11]
- **Soft airbrush:** `samples + held-time ticks → rate emitter → soft falloff mask → low-flow deposit`; expose softness, build-up per second, size and cap. A path-distance emitter alone cannot paint while stationary, as Krita's Airbrush option, MyPaint's per-second dabs and Photoshop's airbrush behavior demonstrate. Brush overlap and compositing law determine whether holding still saturates, so match neither application's numerical controls by name only. [3][5][9]
- **Graphite/pencil:** `samples → seeded arc-length emitter → pressure/tilt(size, contact) → sparse gaussian/irregular grain → document-fixed paper mask → deposit`. Expose core width, grain, density, pressure response and shading angle. Preserve Painter's existing deterministic pencil splats as a reference output; tilt shading, pigment accumulation and paper tooth would be new visual approximations, and a device with no tilt cannot supply physical angle. [5][9][11]
- **Textured dry brush / dual brush:** `samples → mark emitter → tip A × document-fixed or moving grain → intersection with independently spaced/scattered tip B mask → deposit`. Offer texture depth, grain scale/anchoring, secondary size/count/spacing and combine mode. Photoshop-style intersected tips and Procreate-style independently blended brush outputs require **different** compositing operators; "dual brush" is not a single portable node. Brush-local texture repeated per mark differs from a document-anchored paper field. [2][11][12]
- **Spray / particle:** `samples (+ time?) → emission rate → seeded spatial distribution → tiny tip masks → per-particle color/size → deposit`. Expose number or density, radial/angular distribution, spread, color variation and whether a held pen emits. Krita documents

distinct count/density and radial distributions; a gaussian point cloud can visually approximate spray, but emission statistics and per-particle blending must be specified to simulate it. [8][9]

- **Clone / erase:** `samples → spacing + mask → translated SampleImage(frozen source revision) → colored marks` for clone; `samples → mask → erase-intent operations` for eraser. Expose source point/offset, source revision and source layer versus composite; erase must express document compositing semantics, not just emit white marks. A sampled raster projection can produce a visual clone at one resolution; faithfully editable, resolution-independent cloning needs a source dependency and replay/invalidation policy. Krita's clone source may be a layer or projection. [7]
- **Wet mixer / oil-like:** `samples → close-spaced dabs → sample prior image → update pigment reservoir and pickup → deposit/pull → next state`. Expose wetness, load/charge, canvas mix, pickup radius, depletion, smear/dull mode, and reset at stroke start. Photoshop's two wells, Krita's smearing versus dulling and Procreate's charge/pull motivate **different** state models; a composited translucent trail is a visual approximation, not an equivalent simulation of any of them. Raster feedback versus arbitrary-resolution immutable strokes remains the largest unresolved architectural issue. [3][6][11]

Artist-facing consequences and unanswered comparisons

Proposed interaction: start from named brush-family presets rather than an empty graph. Give each macro a small rack of meaningful controls (size, pressure, opacity, texture, scatter or wetness as appropriate), an optional "open internals" action, and previews of the input path, emitted events, tip mask, grain coordinates, resulting marks and composite. Show whether a sensor is available, which controls vary once per stroke versus per dab, and whether the brush reads prior canvas content. A broken dependency should be visible, not silently replaced by the current background. Keep preview and saved strokes tied to a graph version, asset hashes, random seed and explicitly chosen color/compositing policy; this is a proposed reproducibility requirement, not a feature already present in Painter.

Questions for prototypes rather than decisions here: Does an ink macro feel responsive while producing a deterministic, editable, resolution-independent stroke? Is fixed-paper texture stable under tile zoom/recomputation? Does a time-based airbrush reproduce stationary build-up without nondeterministic input-event counts? Can a clone or mixer with image reads be replayed after editing earlier strokes without unbounded rerendering? How much of a complex graph should an artist see by default? Compare these outcomes against a monolithic native engine, a macro over typed primitives and a pixel-field implementation on the same example strokes before promising universal brush equivalence.

Primary sources (numbered labels used above)

[1] Adobe, "Photoshop brush dynamics reference" (Shape, Color, Transfer): helpx.adobe.com/photoshop/desktop/apply-painting-techniques/fill-objects-selections-layers/brush-dynamics-settings.html.

[2] Adobe, "Texture and dual brush settings": helpx.adobe.com/photoshop/desktop/apply-painting-techniques/brushes-presets/textured-and-dual-brush-settings.html.

[3] Adobe, "Paint with the Mixer Brush": helpx.adobe.com/photoshop/desktop/apply-painting-techniques/create-fill-with-patterns/paint-with-mixer-brush.html.

[4] Krita Manual, "Brush Engines": docs.krita.org/en/reference_manual/brushes/brush_engines.html.

[5] Krita Manual, "Sensors" and "Options": tablet_sensors.html options.html.

[6] Krita Manual, “Color Smudge Brush Engine”: docs.krita.org/en/reference_manual/brushes/brush_engines/color_smudge_engine.html.

[7] Krita Manual, “Clone Brush Engine”: docs.krita.org/en/reference_manual/brushes/brush_engines/clone_engine.html.

[8] Krita Manual, “Spray Brush Engine”: docs.krita.org/en/reference_manual/brushes/brush_engines/spray_brush_engine.html.

[9] MyPaint/libmypaint, source `brushsettings.json` (inputs and setting descriptions): github.com/mypaint/libmypaint/blob/master/brushsettings.json.

[10] MyPaint project documentation, “Brush Engine”: mypaint.app/en/docs/backend/brush-engine/.

[11] Procreate Handbook, “Brush Studio Settings”: help.procreate.com/procreate/handbook/brushes/brush-studio-settings.

[12] Procreate Handbook, “Dual Brush”: help.procreate.com/procreate/handbook/brushes/dual-brush.

[13] Adobe, “Display the Brush Settings panel and brush options” (Scattering category): helpx.adobe.com/photoshop/desktop/apply-painting-techniques/brushes-presets/display-brush-panel-brush-options.html.